

LISTA DE EXERCÍCIOS 3.6

1. Considere um espaço de endereçamento lógico de oito páginas de 1024 bits cada, o qual mapeado para memória física de 32 *frames*, também de 1024 bits.

a. Quantos bits existem no endereço lógico?

Resposta: Endereço lógico: 13 bits ($2^3 * 2^{10}$)

b. Quantos bits existem no endereço físico?

Resposta: Endereço físico: 15 bits ($2^5 * 2^{10}$)

2. Explique por que o modelo de tabela de páginas de um único nível pode vir a consumir muito espaço em memória principal.

Resposta: O modelo de páginas de um único nível pressupõe: uma tabela de páginas por processo e a tabela de páginas deve ser integralmente carregada em memória. Assim, para um espaço de endereçamento de 32 bits, por exemplo, podemos imaginar o seguinte:

12 bits reservados para deslocamento/displacement (usado diretamente para cálculo do endereço físico real) e 20 bits usados para a localização da página virtual na tabela de páginas. Assim, considerando páginas de 4 bits, a tabela de páginas para um único processo consumirá $4 \times 2^{10} \times 2^{10} = 4\text{Mb}$. Considerando 100 processos, haverá um consumo de 400 Mb para esses processos. Se considerar um espaço de endereçamento de 64 bits, teremos: $4 \times 2^2 \times 2^{10} \times 2^{10} \times 2^{10} \times 2^{10} \times 2^{10} = 16\text{Pb}$ para um único processo.

3. Como o modelo de tabela de páginas hierárquico melhora o modelo de tabela de páginas de um único nível?

Resposta: o modelo hierárquico permite quebrar a tabela de páginas de um único nível em vários pedaços, de modo a manter algumas partes em memória principal e outras não. De certa forma, permite “paginar” a tabela de páginas. Por exemplo, em um espaço de endereçamento de 32 bits, com 12 bits reservados para deslocamento/displacement e 20 bits distribuídos em 10 bits para 2 níveis de tabelas de páginas, com 4 bits reservados para o tamanho de páginas, teríamos algo como: tabela interna (com acesso ao *page frame*) de tamanho $4 \times 2^{10} = 4\text{ kb}$. Tabela externa, a qual pode indexar até 1024 tabelas internas, ocupando $4 \times 2^{10} = 4\text{ kb}$. Para realizar a tradução de endereços, um processo precisa no mínimo da tabela externa (4kb) e de uma tabela interna (4kb), totalizando 8 kb. Para 100 processos, teríamos 800kb (muito menor do que os 400 Mb exigidos pelo modelo de um único nível).

4. Como o modelo de tabela de páginas por *hash* funciona?

Resposta: o modelo de tabela de páginas por *hash* utiliza uma função *hash* para gerar a chave de busca na tabela de páginas. Assim, em um cenário onde se reservam 12 bits para deslocamento/displacement e 20 bits para a página virtual, esses 20 bits seriam usados como entrada para uma função *hash* que geraria uma chave de busca para a tabela de páginas por *hash*. Essa chave de busca pode sofrer colisão, o que levaria a criação de uma lista encadeada, na qual cada elemento conteria: o índice de página virtual, o *page frame* e o ponteiro para a próxima página que também foi mapeada para essa chave de busca. Páginas inválidas não constarão nessa lista ligada e precisam ser carregadas a partir do disco. Esse modelo de tabela de páginas tem o benefício de não mapear páginas inválidas na estrutura de lista ligada, o que minimiza o consumo de memória ocupado pela tabela de páginas. Caso seja necessário reduzir ainda mais o consumo de memória, pode-se “paginar” a tabela de páginas, criando-se mais um nível de tabela de páginas, evitando-se com isso, carregar todas as entradas da tabela de página em memória (i.e. o segundo

nível poderia sofrer *swap out*).

5. Como o modelo de tabela de páginas invertida funciona? Trace em sua resposta um paralelo entre o consumo de memória, comparando-o ao modelo de tabela de páginas por *hash* e hierárquico.

Resposta: no modelo de tabela de páginas invertida mapeiam-se as páginas que estão efetivamente carregadas em memória física para a tabela de páginas. Essa característica é similar ao comportamento da tabela de páginas por *hash* tendo em vista que essa só mapeia tabelas de página válidas, mas consegue ser melhor, tendo em vista que no modelo de *hash* tradicional são mapeadas todas as entradas de resultado da função *hash* (independente de se ter páginas válidas presentes ou não). É muito mais econômico que o modelo de páginas hierárquico pelo mesmo motivo. No modelo hierárquico, caso o processo seja composto por 100 páginas, as 100 entradas representativas existem independente de serem válidas no momento ou não. Para conseguir esse resultado, o modelo de tabela de páginas invertido utiliza o PID do processo como parte da informação de indexação. Da mesma forma que nos modelos anteriores, o deslocamento/displacement é reservado diretamente para o cálculo do endereço físico e os demais bits são divididos entre PID e página virtual. Ao se encontrar a entrada, utiliza-se a posição desta na tabela de páginas como *page frame* para o cálculo do endereço físico da página em memória principal. Esse modelo é denominado invertido porque o *page frame* não está diretamente inserido na tabela de páginas. Este é indiretamente obtido por meio da posição da entrada na tabela de páginas.